COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 6, 2026
NAME:	Acelajado, John Vince D. Bienvenido Pangilinan Jr.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 [ACTIVITY NAME] Basic Algorithm Analysis

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

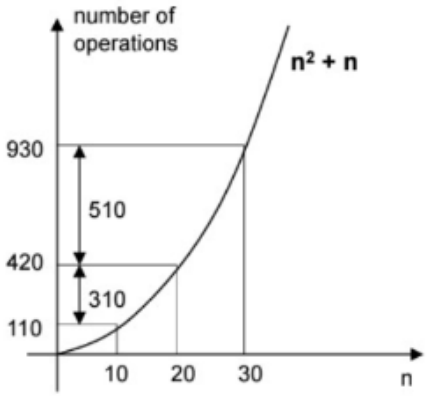
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **m x n**

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

- The graph shows that as the array size (n) grow bigger the the required operations increase much faster because of the formula n^2 + n

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	0 microseconds	`Time taken to search = 0 microseconds` `(525 982) needs vaccination.`	Based on our observation that smaller input size will make the execution speed faster like in the example.
10000	1 microseconds	`Time taken to search = 1 microseconds` `(525 982) needs vaccination.`	This input size is at the middle of the input size of both which make the execution speed at the middle of it.
100000	4 microseconds	`Time taken to search = 4 microseconds` `(525 982) needs vaccination.`	Making the input size bigger will make the execution speed process more slowly.

Analysis:

- The time to execute the program went up with the size of the list, from 0 microseconds for 1000 elements to 1 microsecond for 10000 elements and 4 microseconds for 100000 elements, but very slightly even though the list grew by 100 times. This is because that the program takes binary search, which involves halving the list at each step, and a larger list just increases a few steps. Therefore this matches the theory behind a binary search being O(log n). The student's name, 525 982, was not found in the list and the program returned that the student should be vaccinated.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

PROBLEM 1

CODE

```cpp
#include <iostream>
#include <chrono>
#include <numeric>
#include "problem1.h"

int main() {
    std::cout << "Worst case (all elements unique)\n";
    for (int n : {1000, 2000, 4000, 8000, 16000}) {
        std::vector<int> A(n);
        std::iota(first: A.begin(), last: A.end(), value: 0);

        comparisons = 0;
        auto begin :time_point<steady_clock> = std::chrono::steady_clock::now();
        bool result = unique_elements(A);
        auto end :time_point<steady_clock> = std::chrono::steady_clock::now();

        auto us :long long = std::chrono::duration_cast<std::chrono::microseconds>(d:end - begin).count();
        std::cout << "n = " << n
                  << " | unique = " << result
                  << " | comparisons = " << comparisons
                  << " | n(n-1)/2 = " << (long long)n * (n - 1) / 2
                  << " | time = " << us << " us\n";
    }

    std::cout << "\nBest case (first two elements equal)\n";
    for (int n : {1000, 16000}) {
        std::vector<int> A(n);
        std::iota(first: A.begin(), last: A.end(), value: 0);
        A[1] = A[0];

        comparisons = 0;
        unique_elements(A);
        std::cout << "n = " << n << " | comparisons = " << comparisons << "\n";
    }
    return 0;
}
```

THEORETICAL ANALYSIS

The Unique algorithm checks if all the elements of an array are different. It compares the first element to all elements that follow it, the second element to all elements that follow it, etc. If none of the elements are equal, the worst case scenario occurs: the algorithm has to compare each and every pair, resulting in no match. The first one will be compared n-1 times, the second one will be compared n-2 times, and so on until the last one is compared once. So there will be a total of n(n-1)/2 comparisons. It's nested, so the worst case is $O(n^2)$. Best case, first two elements are the same, algorithm halts immediately after 1 comparison O(1).

EXPERIMENTAL ANALYSIS

```
Worst case (all elements unique)
n = 1000 | unique = 1 | comparisons = 499500 | n(n-1)/2 = 499500 | time = 10106 us
n = 2000 | unique = 1 | comparisons = 1999000 | n(n-1)/2 = 1999000 | time = 17656 us
n = 4000 | unique = 1 | comparisons = 7998000 | n(n-1)/2 = 7998000 | time = 36751 us
n = 8000 | unique = 1 | comparisons = 31996000 | n(n-1)/2 = 31996000 | time = 142290 us
n = 16000 | unique = 1 | comparisons = 127992000 | n(n-1)/2 = 127992000 | time = 667246 us

Best case (first two elements equal)
n = 1000 | comparisons = 1
n = 16000 | comparisons = 1

Process finished with exit code 0
```

ANALYSIS AND COMPARISON

The experiment is in agreement with the theory. For each size, the number of comparisons was n(n-1)/2. Each time n was doubled, the time took about 4 times as long, characteristic of a quadratic algorithm. This proves that Unique is $O(n^2)$ worst case. Regardless of how large the data array was, the best case took only 1 comparison, so the data in the data array also has some influence on the speed, not only size. This means that Unique is OK when your array is small, but is very slow if your array is large.

PROBLEM 2

CODE

```cpp
#include <iostream>
#include <chrono>
#include <iomanip>
using namespace std;
using namespace chrono;

long long calls = 0;  // counts recursive calls (theoretical operation count)

// Algorithm rpower: O(n)
long long rpower(long long x, int n) {
    calls++;
    if (n == 0) return 1;
    return x * rpower(x, n - 1);
}

// Algorithm brpower: O(log n)
long long brpower(long long x, int n) {
    calls++;
    if (n == 0) return 1;
    if (n % 2 == 1) {
        long long y = brpower(x, (n - 1) / 2);
        return x * y * y;
    } else {
        long long y = brpower(x, n / 2);
        return y * y;
    }
}

int main() {
    // x = 1 so large exponents don't overflow; call counts and timing are unaffected.
    int sizes[] = {10, 100, 1000, 5000, 10000};
    const int REPS = 1000;  // repeat to get measurable time

    cout << left << setw(8) << "n" << setw(16) << "rpower calls"
         << setw(16) << "brpower calls" << setw(18) << "rpower time(ns)"
         << "brpower time(ns)" << "\n";
    for (int n : sizes) {
        calls = 0; rpower(1, n);  long long c1 = calls;
        calls = 0; brpower(1, n); long long c2 = calls;

        volatile long long sink;
        auto b1 = steady_clock::now();
        for (int r = 0; r < REPS; r++) sink = rpower(1, n);
        auto e1 = steady_clock::now();

        auto b2 = steady_clock::now();
        for (int r = 0; r < REPS; r++) sink = brpower(1, n);
        auto e2 = steady_clock::now();

        cout << left << setw(8) << n << setw(16) << c1 << setw(16) << c2
             << setw(18) << duration_cast<nanoseconds>(e1 - b1).count() / REPS
             << duration_cast<nanoseconds>(e2 - b2).count() / REPS << "\n";
    }
    return 0;
}
```

THEORETICAL ANALYSIS

For rpower, each recursive call decreases n by 1 for example n,n-1,n-2,...so on, and performs one multiplication, so it will make n+1 recursive calls. Because the amount of work per call remains constant, the running time increases by a fixed amount for each call, resulting in the running time being proportional to n, hence rpower is O(n). As a case in point: With n = 16, it calls 17 times.

With brpower, each recursive call reduces n by half for example n, n/2, n/4, …so on, and does a single recursive call per level since the result is stored in y and used later as y*y. The function calls itself approximately $\log_2 n + 2$ times which means brpower has run time of O(log n). If n = 16, for instance, it will only make 6 calls.

EXPERIMENTAL ANALYSIS

```
[Running] cd "c:\Users\My PC\Downloads\ACELAJADO C++\HOA11\" && g++ problem2.cp
n       rpower calls    brpower calls   rpower time(ns)   brpower time(ns)
10      11              5               16                10
100     101             8               178               17
1000    1001            11              1747              31
5000    5001            14              8783              34
10000   10001           15              17939             36

[Done] exited with code=0 in 1.379 seconds
```

ANALYSIS AND COMPARISON

The experimental results are in good agreement with the theoretical analysis. For rpower, the number of calls is always n + 1, which gave 11, 101, 1001, 5001 and 10001 in the output, and the time grew about 10 times whenever n grew 10 times, from 178 ns to 1,747 ns, so it is linear, or O(n). For brpower, the number of calls is $\lfloor \log_2 n \rfloor + 2$, which grew by 5, 8, 11, 14 and 15 when n increased by 1,000 times, making it logarithmic or O(log n). This is because rpower subtracts 1 from n each time it is called, but brpower halves n each time it is called. For n = 10,000, rpower took 17,939 ns while brpower took 36 ns, and in fact brpower is much more efficient for large n, though the results are close for very small n.

CONCLUSION

Acelajado

- During this lab I learned that there are two ways of checking an algorithm by examining the steps on paper and by actually executing it and measuring the time. The answers were the same for both directions. I discovered that if an algorithm has a loop inside a loop, such as Unique and Diff, then as

the input increases, it becomes very slow very rapidly, so only useful for small sized inputs. I also discovered some algorithms that divide the problem into two halves each time, such as binary search and brpower, remain fast even if the input increases in size. The rpower function only reduces the problem by 1 step at a time, thus the more the input becomes, the more time this takes. This gave me the idea that the growth of the algorithm is more important than the exact speed of the algorithm, and that the selection of a better algorithm has a significant impact on the large inputs.

Pangilinan

- Throughout this entire activity, in the first part the is the counting comparisons in the different function showed that two loops, one inside and the other give the m x n comparisons and once the both arrays are in the same size the graph of n^2 + n climbs higher and higher from 110, 420, to 930 and also In the binary search test, making the list 100 times bigger only moved the time from 0 to 4 microseconds, because each step cuts the remaining list in half. In the Unique problem, the comparisons matched n(n-1)/2 exactly, and the time roughly quadrupled each time n was doubled. In the power problem, rpower needed n + 1 calls while brpower needed only about log n calls, which is why 10,000 took thousands of nanoseconds for one and only a few dozen for the other.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Use for Grammar in some explanation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY